LIVE WORKSHOP · 60 MIN

Introduction to subagents with Claude

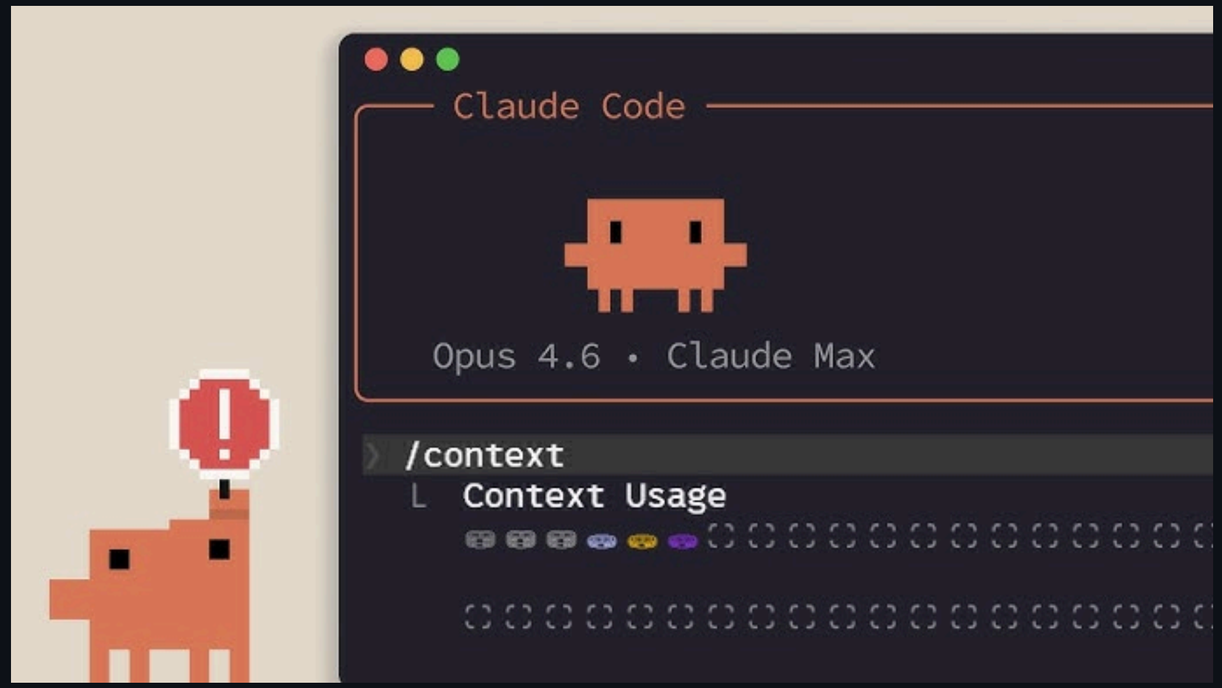
Patterns on a real repo: djobmatch

Sergio Infante

THE PROBLEM

In long sessions, context gets dirty

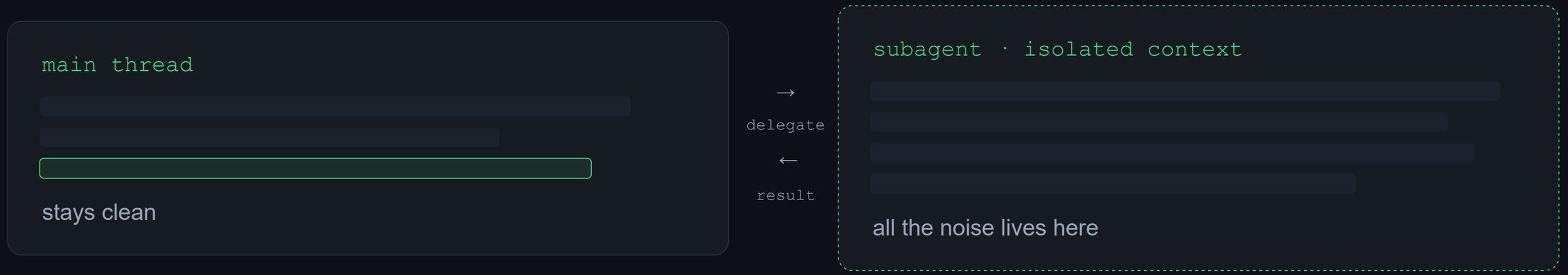
- Every exploration, log and stack trace piles up
- The main thread fills with noise
- Claude starts losing focus on the task
- More tokens at play, worse answers



THE IDEA

A helper that works on the side

It works in its own context and returns only the result.



Like browser tabs: you open one, do the work, come back with the answer.

THE EXAMPLE REPO

djobmatch

A REST API in Django + Django Ninja. Parses jobs and resumes with an LLM and scores the match by cosine with pgvector.

ITS SEAMS

- services/ as ports and adapters
- A cache invariant to respect
- Endpoints never call the SDK directly

```
djobmatch/
├─ api/ # Ninja endpoints
├─ services/ # ports/adapters
│ ├─ llm.py
│ └─ scoring.py
├─ models/ # pgvector
└─ tests/
```

MODULE 1 · WHAT ARE THEY?

A subagent is an isolated agent

01

Its own context

a separate window from the main thread

02

Its system prompt

instructions dedicated to one task

03

Its tools and model

you choose what it can touch and what it runs on

04

Returns what's relevant

only the result comes back to the main thread

The built-ins you already use

Explore

Fast, read-only codebase search. Runs on Haiku by default — quick and cheap.

Plan

Used in plan mode. Researches the codebase in a separate context before Claude proposes a strategy.

General-purpose

Tasks that need both exploration AND changes, or several dependent steps.

statusline-setup

configures your status line

Claude Code Guide

answers questions about Claude Code

"The pattern already runs under the hood. Custom subagents just let you build your own."

Three wins

01

Clean context

The noise stays out of the main thread.

02

Parallelism

Several subagents working at once.

03

Lower cost

Restrict tools and use Haiku for read-only tasks.

MODULE 2 · CREATE

/agents

- Library and Running tabs
- “Generate with Claude” from a description
- Choose tools — read-only option
- Choose model (Haiku · Sonnet · Opus)

```
$ claude
> /agents
# [ Library ] Running
+ Create new agent
Generate with Claude
```

Anatomy of a subagent

- Markdown with YAML frontmatter
- The body = system prompt
- .claude/agents/ project · shared
- ~/.claude/agents/ user-level

```
---
name: code-reviewer
description: >-
Reviews changes. Use proactively
after editing services/.
tools: Read, Grep, Glob
model: sonnet
---

You are a senior code reviewer.
Review ONLY the changed files and
return a prioritized list.
```

MODULE 2 · CREATE

[LIVE DEMO]

Create the code-reviewer

over djobmatch, live

$ claude

MODULE 3 · DESIGN · PRINCIPLE 1

The description drives delegation

Claude decides when to delegate by reading the description.

✗ vague

```
Reviews code.
```

✓ explicit

```
Reviews changes. Use proactively after editing
services/.
```

Key phrases: use proactively · MUST BE USED

Structured output

Ask for a prioritized list of findings, not a paragraph. Compact and actionable.

```
# Findings (by severity)
[HIGH] endpoint calls the SDK
[MED] cache not invalidated
[LOW] naming in scoring.py
```

MODULE 3 · DESIGN · PRINCIPLE 3

Report obstacles, don't invent

✗ invents

Fills gaps with an answer that looks correct.

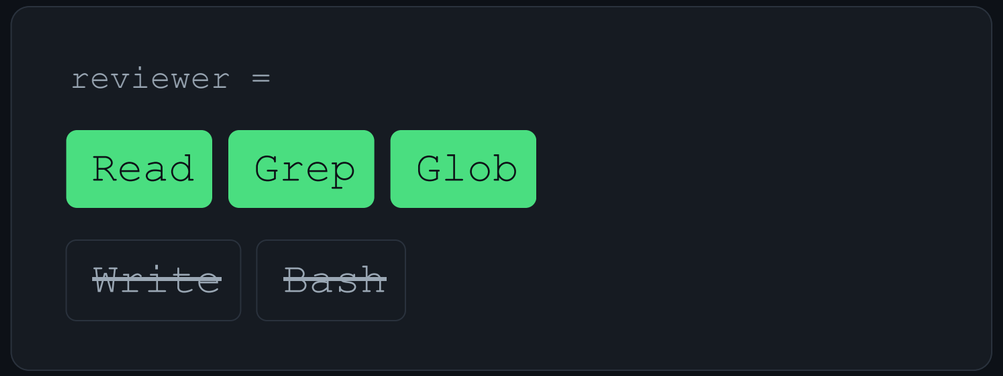
✓ reports

“Couldn't review X: missing context from scoring.py.”

A reported obstacle gets resolved. A hallucination propagates.

Limit the tools

- More focus on the task
- Less risk surface
- Enables a cheaper model



When to use them

- ✓ Investigate or explore a large repo
- ✓ Parallelize independent work
- ✓ Fresh review, free of the thread's bias

When not to

- ✗ When the overhead isn't worth it (trivial task)
- ✗ When you need the context in the main thread
- ✗ When the work is interactive, step by step

Chain them, don't pile up

`reviewer`
list of findings

→
passes its output

`fixer`
applies the changes

`anti-pattern` one subagent that reviews, fixes, tests and writes — it dirties its own context again.

CLOSING · DESIGN BY SEAMS

The 4 subagents of djobmatch

SUBAGENT	TOOLS	MODEL
architecture-explorer	Read · Grep · Glob	Haiku · read-only
code-reviewer	Read · Grep · Glob	Sonnet · read-only
test-writer	Read · Write · Bash	Sonnet
provider-adapter	Read · Write · Edit	Sonnet

Pick the model by how much thinking it needs

OPUS Open-ended judgment: no single right answer.

Agent: `matching-strategist` · reserve for reasoning-heavy work.

SONNET Structured production: review, test, scaffold.

`code-reviewer` · `test-writer` · `provider-adapter` · the default workhorse.

HAIKU Mechanical work: search, map, summarize.

Agent: `architecture-explorer` · cheap and fast.

> **Choose the model by how much thinking the task needs; choose the tools by how much it can break.**

Don't default to Opus — it's slower and pricier. Use `model: inherit` to follow the main session's model.

The 5 subagents of djobmatch

SUBAGENT	TOOLS	MODEL
architecture-explorer	Read · Grep · Glob	Haiku · read-only
code-reviewer	Read · Grep · Glob	Sonnet · read-only
test-writer	Read · Write · Bash	Sonnet
provider-adapter	Read · Write · Edit	Sonnet
matching-strategist	Read · Grep · Glob	Opus · read-only

CLOSING

Recap & resources

TO REMEMBER

- Design around the repo's seams
- Isolated context, structured output
- The description drives delegation
- Limit tools, cheapen the model

RESOURCES

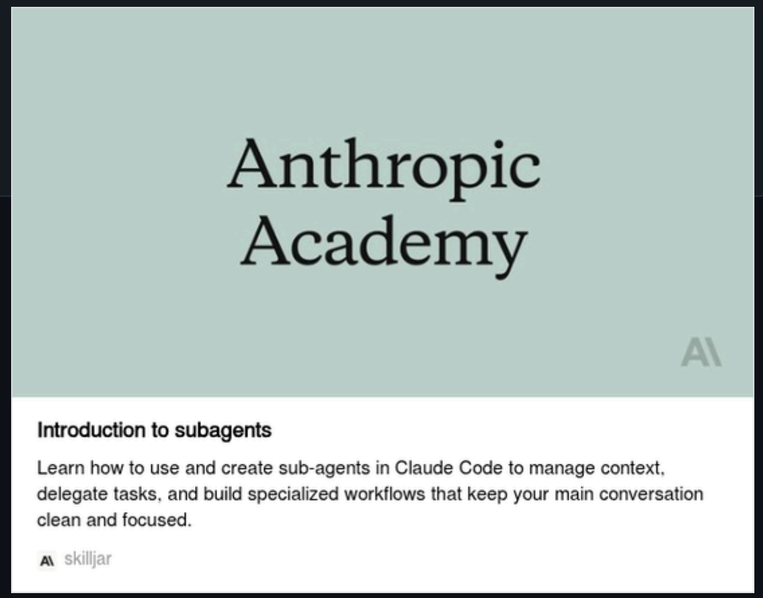
djobmatch

the example repo, to practice

https://github.com/neosergio/djobmatch

Introduction to subagents

course by Anthropic



THANKS

Happy hacking :)

Sergio Infante

neosergio.net · github.com/neosergio